Федеральное государственное автономное образовательное учреждение
высшего образования
«Национальный исследовательский университет ИТМО»

Факультет программной инженерии и компьютерной техники

Лабораторная работа №1
«Модульное тестирование»

по дисциплине «Тестирование программного обеспечения»

Вариант: 413732

Студент:
Анкудинов Кирилл Константинович
Группа:
P3318
Преподаватель:
Егошин Алексей Васильевич

Санкт-Петербург, 2026

Оглавление

1. Задание 1. Разложение $\cos(x)$ в степенной ряд . . . 2
   1.1. Описание реализации . . . 2
   1.2. Методика . . . 2
   1.3. Результаты . . . 3
2. Задание 2. Биномиальная куча . . . 4
   2.1. Описание реализации . . . 4
   2.2. Методика тестирования . . . 4
      2.2.1. Таблицы решений для `insert` . . . 4
      2.2.2. Таблицы решений для `extractMin` . . . 4
   2.3. Результаты . . . 4
3. Задание 3. Доменная модель . . . 6
   3.1. Исходный текст . . . 6
   3.2. Доменная модель . . . 6
   3.3. Методика тестирования . . . 6
      3.3.1. Таблицы переходов `Person` . . . 6
      3.3.2. Таблицы переходов `Child` . . . 8
      3.3.3. Таблицы переходов `Horse` . . . 9
      3.3.4. Таблицы переходов `Supply` . . . 10
   3.4. Результаты . . . 11
4. Заключение . . . 11

1. Задание 1. Разложение $\cos(x)$ в степенной ряд

1.1. Описание реализации

Функция $\cos(x)$ вычисляется через ряд Тейлора:

$$\cos(x) = \sum_{n=0}^{\infty} \frac{(-1)^n x^{2n}}{(2n)!} = 1 - \frac{x^2}{2!} + \frac{x^4}{4!} - ...$$

Перед вычислением аргумент приводится к диапазону $[-\pi, \pi]$. Суммирование продолжается до тех пор, пока абсолютное значение очередного слагаемого не станет меньше `eps` (по-умолчанию - `Double.MIN_VALUE`).

1.2. Методика

Тестирование проводится при помощи **анализа эквивалентности**.

$$\frac{d}{dx}\cos(x) = -\sin(x) = 0 \Rightarrow x = \pi n, n \in \mathbb{Z}$$

$$\frac{d^2}{dx^2}\cos(x) = -\cos(x) = 0 \Rightarrow \frac{\pi}{2} + \pi n, n \in \mathbb{Z}$$

- Граничные значения:
  1. $x = 0$
  2. $x = \frac{\pi}{2}$
  3. $x = \pi$
  4. $x = \frac{3\pi}{2}$
  5. $x = 2\pi$
- Граничные значения (с точки зрения языка прогрммирования):
  1. $x = \mathrm{NaN}$
  2. $x = \pm\infty$
- Области эквивалентности:
  1. $\left(0; \frac{\pi}{2}\right)$
  2. $\left(\frac{\pi}{2}; \pi\right)$
  3. $\left(\pi; \frac{3\pi}{2}\right)$
  4. $\left(\frac{3\pi}{2}; 2\pi\right)$

Note: $\cos(x)$ — функция с периодом 2π

Тестовое покрытие:

Аргумент $\cos(x)$	Ожидаемый результат	Граничное значение?
$x = \mathrm{NaN}$	NaN	✓
$x = \pm\infty$	NaN	✓
$x = 0$	1	✓
$x = \frac{\pi}{4}$	$0,7071067812$	✗
$x = \frac{\pi}{2}$	0	✓
$x = \frac{3\pi}{4}$	$-0,7071067812$	✗
$x = \pi$	-1	✓
$x = \frac{5\pi}{4}$	$-0,7071067812$	✗
$x = \frac{3\pi}{2}$	0	✓
$x = \frac{7\pi}{4}$	$0,7071067812$	✗
$x = 2\pi$	1	✓

1.3. Результаты

```
> Task :task1:test
cos(x) to Tailor series > Boundaries > cos(NaN) = NaN PASSED
cos(x) to Tailor series > Boundaries > cos(Infinity) = NaN PASSED
cos(x) to Tailor series > Boundaries > cos(-Infinity) = NaN PASSED
cos(x) to Tailor series > Boundaries > cos(3.141592653589793) = -1
PASSED
cos(x) to Tailor series > Boundaries > cos(-3.141592653589793) = -1
PASSED
cos(x) to Tailor series > Equivalence classes > cos(10.0) =
-0.8390715291 PASSED
cos(x) to Tailor series > Equivalence classes > cos(0.0) = 1.0
PASSED
cos(x) to Tailor series > Equivalence classes > cos(-1.0) =
0.5403023059 PASSED
```

Все тесты пройдены успешно.

2. Задание 2. Биномиальная куча

2.1. Описание реализации

Биномиальная куча (Binomial Heap) — структура данных, представляющая собой набор биномиальных деревьев, упорядоченных по степени. Реализованы ключевые методы:

- `insert`
- `extractMin`

2.2. Методика тестирования

Тестирование проводится при помощи **таблицы решений**.

Условия — **наличие/отсутствие** деревьев рангов 0, 1, 2 в корневом списке до операции. Действия — проверка соответствия внутреннего состояния теоретическому либо ожидание исключения.

2.2.1. Таблицы решений для `insert`

Таблица решений для `insert`	T1	T2	T3	T4	T5	T6	T7	T8
Условия								
$\exists B_0$		+			+	+		+
$\exists B_1$			+		+		+	+
$\exists B_2$				+		+	+	+
Действия								
Состояние = теоретическое	+	+	+	+	+	+	+	+

2.2.2. Таблицы решений для `extractMin`

Таблица решений для `extractMin`	T1	T2	T3	T4	T5	T6	T7	T8
Условия								
$\exists B_0$		+			+	+		+
$\exists B_1$			+		+		+	+
$\exists B_2$				+		+	+	+
Действия								
Ожидаем исключение	+							
Корректное значение min		+	+	+	+	+	+	+
Состояние = теоретическое		+	+	+	+	+	+	+

2.3. Результаты

```
> Task :task2:test
Binomial Heap > extractMinon heap with one element with reverse
```

```
PASSED
Binomial Heap > Correctness of operations - several inserts and
extractMins PASSED
Binomial Heap > Merge of two T0 PASSED
Binomial Heap > Exception when extractMin on empty heap PASSED
Binomial Heap > getMin on heap with one element PASSED
Binomial Heap > extractMin on heap with one element PASSED
Binomial Heap > Exception when getMin on empty heap PASSED
Binomial Heap > Merge of T1 and T0 PASSED
Binomial Heap > Insert into empty heap PASSED
```

Все 9 тестов пройдены успешно. Неудачных тестов нет.

3. Задание 3. Доменная модель

3.1. Исходный текст

> *«Они сидели на мостовой и смотрели с некоторым беспокойством, как огромные дети тяжело прыгают по песку, а дикие лошади с грохотом везут по небу в Неизведанные Области свежие запасы армированных изгородей.»*

3.2. Доменная модель

Из текста выделены следующие сущности и их отношения:

#TODO: insert UML diagram

3.3. Методика тестирования

Тестирование проводится при помощи **таблицы переходов**.

3.3.1. Таблицы переходов `Person`

Таблица `watch`:

№	Текущее состояние	Вход / условие	Следующее состояние	Ожидаемый результат
1	(нет объекта)	`init`	`CALM`	`emotion == CALM`
2	`ANY`	`watchChild(size = HUGE)`	`SOMEWHAT_WORRIED`	`emotion == SOMEWHAT_WORRIED`
3	`ANY`	`watchChild(size = NORMAL)`	`CALM`	`emotion == CALM`
4	`ANY`	`watchHorse(wild = true)`	`WORRIED`	`emotion == WORRIED`
5	`ANY`	`watch(null)`	(исключение)	`IllegalArgumentException`

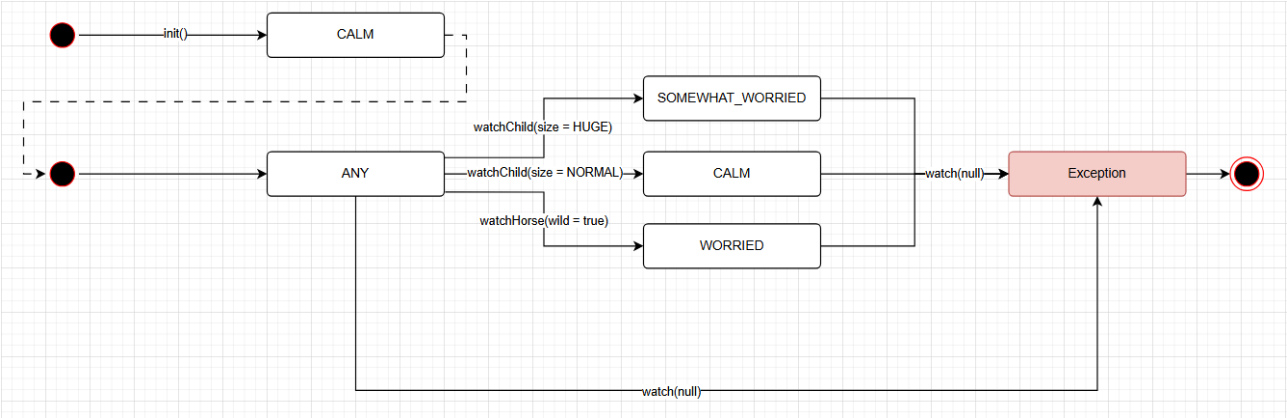

Рис. 1. Конечный автомат эмоций `Person`

Таблица `sitOn`:

№	Текущее состояние	Вход / условие	Следующее состояние	Ожидаемый результат
1	(нет объекта)	`init()`	`NO_LOCATION`	`location == null`
2	`NO_LOCATION`	`sitOn(location != null)`	`AT_LOCATION(L)`	`location == L`
3	`NO_LOCATION`	`sitOn(null)`	(исключение)	`IllegalArgumentException("Location cannot be null")`
4	`AT_LOCATION(L)`	`sitOn(location == L)`	(исключение)	`IllegalArgumentException("Person is already at this location")`
5	`AT_LOCATION(L1)`	`sitOn(location = L2), L2 != null, L2 != L1`	`AT_LOCATION(L2)`	`location == L2`

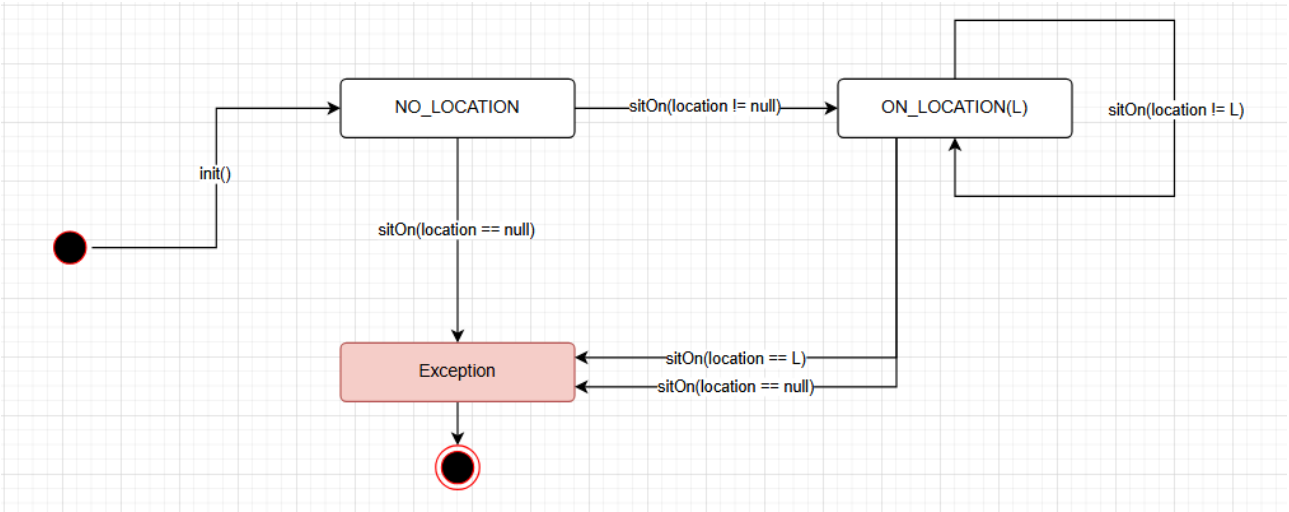

Рис. 2. Конечный автомат местоположений `Person`

3.3.2. Таблицы переходов `Child`

№	Текущее состояние	Вход / условие	Следующее состояние	Ожидаемый результат
1	(нет объекта)	`init(size != null)`	`NO_LOCATION`	`== null`, размер сохранён
2	(нет объекта)	`init(size == null)`	(исключение)	`IllegalArgumentException("Size cannot be null")`
3	`NO_LOCATION`	`jumpOn(sand != null)`	`ON_SAND`	`location == sand`
4	`NO_LOCATION`	`jumpOn(sand == null)`	(исключение)	`IllegalArgumentException("Sand cannot be null")`
5	`ON_SAND`	`jumpOn(sand != null)`	`ON_SAND`	Локация меняется на новый `sand`

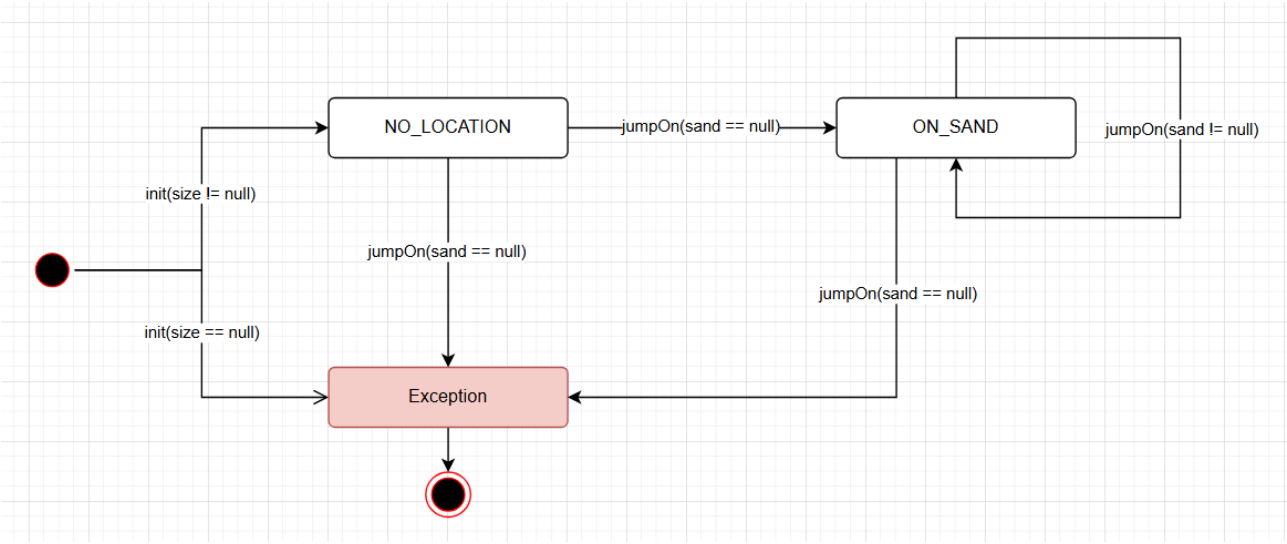

Рис. 3. Конечный автомат прыжков `Child` на песке

3.3.3. Таблицы переходов `Horse`

№	Текущее состояние	Вход / условие	Следующее состояние	Ожидаемый результат
1	(нет объекта)	`init(wild)`	`NO_DESTINATION`	`location == null`
2	`NO_DESTINATION`	`pull(cargo != null, sky != null, destination != null)`	`AT_UNKNOWN_REGIONS`	`location == destination`
3	`NO_DESTINATION`	`pull(cargo == null, sky != null, destination != null)`	(исключение)	`IllegalArgumentException("Cargo cannot be null")`
4	`NO_DESTINATION`	`pull(cargo != null, sky == null, destination != null)`	(исключение)	`IllegalArgumentException("'Through' cannot be null")`
5	`NO_DESTINATION`	`pull(cargo != null, sky != null, destination == null)`	(исключение)	`IllegalArgumentException("Destination cannot be null")`

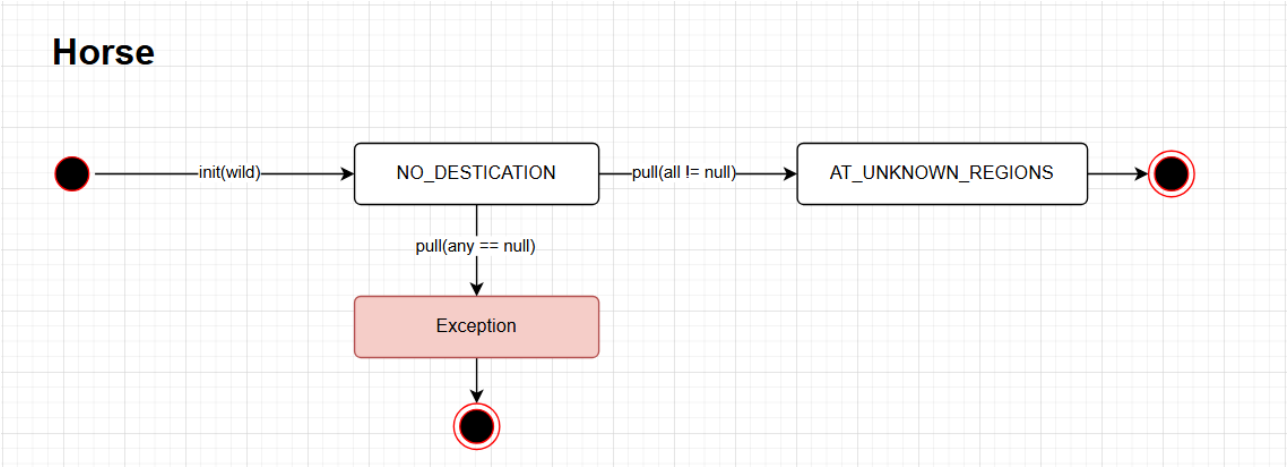

Рис. 4. Конечный автомат местоположений `Horse`

3.3.4. Таблицы переходов `Supply`

№	Текущее состояние	Вход / условие	Следующее состояние	Ожидаемый результат
1	(нет объекта)	`init(fences == null)`	(исключение)	`IllegalArgumentException("Supply must contain at least one fence")`
2	(нет объекта)	`init(fences.isEmpty())`	(исключение)	`IllegalArgumentException("Supply must contain at least one fence")`
3	(нет объекта)	`init(all fences FRESH)`	`FRESH`	`getOverallFreshness() == Freshness.FRESH`
4	(нет объекта)	`init(any fence STALE)`	`STALE`	`getOverallFreshness() == Freshness.STALE`

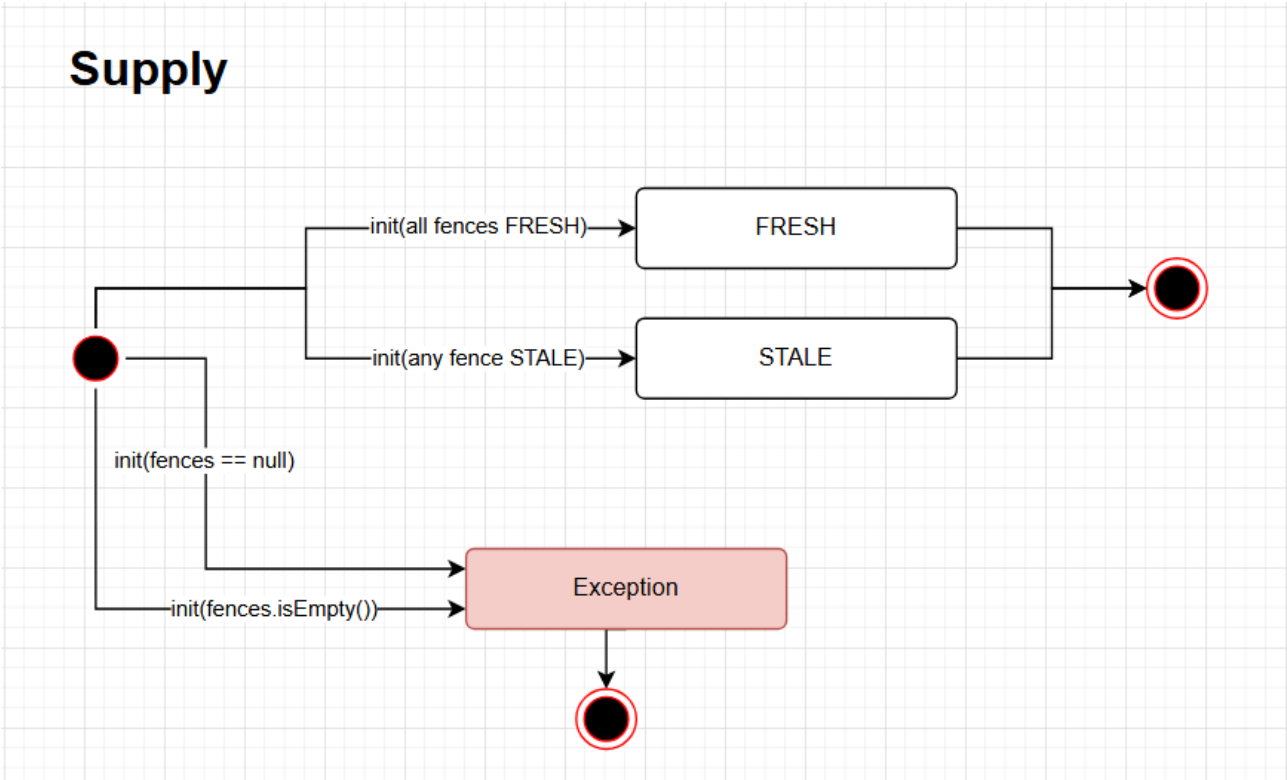

Рис. 5. Конечный автомат свежести `Supply`

3.4. Результаты

```
> Task :task3:test
AppTest > horseHasNoDestinationBeforePulling() PASSED
AppTest > sandIsCalledSand() PASSED
AppTest > horsesAreWild() PASSED
AppTest > worryIsSomeNotExtreme() PASSED
AppTest > childrenAreNotNormalSized() PASSED
AppTest > childrenAreHuge() PASSED
AppTest > fenceIsAlwaysReinforcedRegardlessOfFreshness() PASSED
AppTest > pavementIsCalledPavement() PASSED
AppTest > childrenJumpOnSand() PASSED
AppTest > supplyWithOneStaleFenceIsNotFresh() PASSED
AppTest > watchingHugeChildCausesSomeWorry() PASSED
AppTest > horsesArriveAtUnknownRegions() PASSED
AppTest > routeGoesthroughTheSky() PASSED
AppTest > unknownRegionsIsCalledCorrectly() PASSED
AppTest > childHasNoLocationBeforeJumping() PASSED
AppTest > theyAreOnThePavement() PASSED
AppTest > allFencesInSupplyAreReinforced() PASSED
AppTest > suppliesAreFresh() PASSED
```

Все тесты пройдены успешно. Неудачных тестов нет.

4. Заключение

В ходе лабораторной работы было реализовано и протестировано три программных модуля. Итоговая статистика:

Подпроект	**Всего**	**Пройдено**	**Ошибок**	**Пропущено**
task1 ($\cos(x)$)	29	29	0	0
task2 (Binomial Heap)	9	9	0	0
task3 (Domain model)	18	18	0	0
Итого	**56**	**56**	**0**	**0**

Все 56 тестов пройдены успешно, сбоев нет.